mathematics

MDPI

Article

UAV Path Planning for Forest Firefighting Using Optimized Multi-Objective Jellyfish Search Algorithm

Rui Zeng [1], Runteng Luo [2] and Bin Liu [2,*]

[1] School of Computer Science and School of Cyberspace Science, Xiangtan University, Xiangtan 411105, China; 202205710118@smail.xtu.edu.cn
[2] School of Physics and Optoelectronics, Xiangtan University, Xiangtan 411105, China; 202205710216@smail.xtu.edu.cn
* Correspondence: liubin@xtu.edu.cn

Abstract

This paper presents a novel approach to address the challenges of complex terrain, dynamic wind fields, and multi-objective constraints in multi-UAV collaborative path planning for forest firefighting missions. An extensible algorithm, termed Parallel Vectorized Differential Evolution-based Multi-Objective Jellyfish Search (PVDE-MOJS), is proposed to enhance path planning performance. A comprehensive multi-objective cost function is formulated, incorporating path length, threat avoidance, altitude constraints, path smoothness, and wind effects. Forest-specific constraints are modeled using cylindrical threat zones and segmented wind fields. The conventional jellyfish search algorithm is then enhanced through multi-core parallel fitness evaluation, vectorized non-dominated sorting, and differential evolution-based mutation. These improvements substantially boost convergence efficiency and solution quality in high-dimensional optimization scenarios. Simulation results on the Phillip Archipelago Forest Farm digital elevation model (DEM) in Australia demonstrate that PVDE-MOJS outperforms the original MOJS algorithm in terms of inverted generational distance (IGD) across benchmark functions UF1–UF10. The proposed method achieves effective obstacle avoidance, altitude optimization, and wind adaptation, producing uniformly distributed Pareto fronts. This work offers a viable solution for emergency UAV path planning in forest fire rescue scenarios, with future extensions aimed at dynamic environments and large-scale UAV swarms.

Keywords: multi-UAV collaboration; path planning; jellyfish search algorithm; multi-objective optimization; forest firefighting; wind field modeling

MSC: 93C85

check for updates

Academic Editor: Duarte Valério

Received: 21 July 2025
Revised: 12 August 2025
Accepted: 21 August 2025
Published: 26 August 2025

Citation: Zeng, R.; Luo, R.; Liu, B. UAV Path Planning for Forest Firefighting Using Optimized Multi-Objective Jellyfish Search Algorithm. *Mathematics* **2025**, *13*, 2745. https://doi.org/10.3390/math13172745

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In the context of accelerating global ecological environment construction, forest resources, as the core of terrestrial ecosystems, play an increasingly prominent role in carbon sequestration and biodiversity protection [1,2]. Sustainable forest development is crucial for maintaining ecological balance, driving economic growth, and ensuring livelihoods. However, frequent forest fires severely threaten forest resource safety, becoming a global challenge in forest protection [3]. Traditional firefighting methods are inefficient in complex terrains and harsh environments, while unmanned aerial vehicle (UAV) technology, with its high mobility, efficient operation capabilities, and excellent adaptability to complex environments, opens new avenues for forest firefighting [4,5]. UAVs can quickly monitor fire

situations and precisely deliver firefighting resources, significantly enhancing firefighting efficiency and becoming a key tool in forest fire prevention and control [6].

Although UAVs have significant application potential in forest firefighting, the issue of scheduling optimization still severely restricts their performance [7]. The complex geographical environment, changeable meteorological conditions, and differentiated firefighting task requirements in the forest impose strict demands on UAV path planning and task allocation. Communication interference and flight conflicts during multi-UAV collaborative operations further increase scheduling complexity [8]. Existing path planning algorithms struggle to comprehensively balance the interactions of multiple factors, leading to insufficiently practical scheduling schemes that seriously affect the efficiency of fire emergency response. Therefore, there is an urgent need to develop optimization strategies adapted to complex scenarios to enhance the scientific nature of UAV scheduling and the collaborative effectiveness of firefighting [9,10].

In the field of UAV path planning, traditional methods (e.g., the Dijkstra algorithm [11], A* algorithm [12], and artificial potential field method [13]) and sampling-based methods (such as the probabilistic roadmap algorithm [14] and rapidly-exploring random tree algorithm [15]) often suffer from low computational efficiency and limited capability to handle complex environments and multi-objective constraints [16]. To overcome these technical bottlenecks, meta-heuristic algorithms have gradually become a research focus [17–20]. Among them, optimization algorithms based on swarm intelligence, as an important branch of meta-heuristic algorithms, stand out due to their advantage in modeling the cooperative behavior of biological groups. They demonstrate efficient multi-objective optimization capabilities in complex search spaces [21], with typical representatives including the particle swarm optimization algorithm [20,22], ant colony optimization algorithm [23,24], genetic algorithm [18,25], differential evolution algorithm [19,26], artificial bee colony algorithm [27,28], cuckoo search algorithm [29,30], and simulated annealing algorithm [31,32].

Swarm intelligence-based optimization algorithms draw inspiration from biological group cooperation, such as ants using pheromones for optimal paths, bird flocks maintaining formations during migration, and fish schools evading predators. As computational models simulate biological swarm intelligence, they achieve optimization goals through information interaction and coordination among individuals [33]. Unlike traditional evolutionary algorithms (relying on mutation and crossover), these algorithms guide search via individuals learning from historical experiences and sharing group knowledge. In complex optimization scenarios characterized by high dimensionality, multi-objective trade-offs, and non-convexity—such as the forest firefighting UAV path planning problem addressed herein—swarm intelligence algorithms demonstrate distinct advantages. Unlike gradient-based methods that struggle with non-differentiable constraints and local optima in non-convex spaces, swarm intelligence approaches leverage population diversity to explore broader solution spaces, making them particularly suitable for balancing conflicting objectives [34]. They are robust to initial conditions and changes in the objective function, adapting to scenarios by adjusting interaction coefficients and weights. Despite their potential in AUV path planning, engineering applications face limitations [35]. For example, some algorithms improve convergence efficiency via dynamic parameter adjustment but fail to fully resolve local optima. While multi-algorithm fusion enhances search capability, it complicates the architecture, leading to excessive computational overhead and reduced real-time optimization efficiency. Moreover, in scenarios such as dynamic obstacle avoidance and multi-objective collaborative optimization in complex forest environments, existing algorithms still lack sufficient adaptive precision and environmental robustness.

The traditional Multi-Objective Jellyfish Search algorithm (MOJS), a high-precision, adaptable swarm intelligence method, handles multi-objective constraints in path planning

via biological swarm behavior, balancing convergence and diversity [36]. However, in high-dimensional scenarios, MOJS faces three bottlenecks: serial fitness evaluation (efficiency decline with dimension), low non-dominated sorting (solution set update delay), and fixed strategies (failing dynamic multi-objective collaboration). To address these, this paper proposes PVDE-MOJS, enhancing MOJS with three innovations: multi-core parallel fitness evaluation (linear acceleration), vectorized non-dominated sorting (faster solution screening), and differential evolution mutation (dynamic adaptability). These improve convergence and solution quality in high-dimensional tasks. Experiments show PVDE-MOJS's evaluation calls ($D \times 100$ here is the decision dimension) outperform existing algorithms, validating its engineering potential.

The rest of the paper is organized as follows. Section 2 models UAV path planning (optimal paths, constraints, cost function). Section 3 details PVDE-MOJS (improvements, framework). Section 4 simulates multiple scenarios, explaining spherical vectors and verifying PVDE-MOJS's advantages. Section 5 summarizes results and future work.

2. Formulation of Constraints in the Problem

In forest firefighting scenarios, path planning aims to generate feasible routes for UAVs to travel from predefined starting points to target endpoints (e.g., fire sites) via sequential waypoints, while avoiding obstacles and satisfying operational constraints. A route is defined as a sequence of waypoints $P = \{P_0, P_1, \ldots, P_n\}$, where P_0 denotes the starting point (initial position of the UAV), P_n denotes the endpoint (target fire site), and P_i ($1 \leq i \leq n-1$) are intermediate waypoints. The following sections formalize the optimality criteria, constraints, and cost functions based on this structure.

2.1. Formulation of Optimal Path

In the forest fire extinguishing scenario, to enable UAVs to execute tasks efficiently, the planned path must satisfy specific optimality criteria. Here, the starting point $P_0 = (x_0, y_0, z_0)$ and endpoint $P_n = (x_n, y_n, z_n)$ are fixed 3D coordinates in the forest spatial system (with z as relative altitude above ground). The path length is calculated as the sum of Euclidean distances between consecutive waypoints, from P_0 to P_n, to minimize flight time and energy consumption. Given the complex terrain of forest areas and the high timeliness requirement of fire suppression tasks, minimizing the path length can reduce UAV flight time and improve operational efficiency. Thus, this study selects minimizing the path length as the core objective. The UAV, controlled by a ground station, follows a flight path X_i consisting of n waypoints, which are sequential positions the UAV must traverse during forest fire operations. In the forest spatial coordinate system, each waypoint corresponds to a path node $P_{ij} = (x_{ij}, y_{ij}, z_{ij})$. The Euclidean distance between adjacent nodes is denoted as $|\overrightarrow{P_{ij}P_{i,j+1}}|$. The cost function for path length is formulated as:

$$F_1(X_i) = \sum_{j=1}^{n-1} |\overrightarrow{P_{i,j}P_{i,j+1}}| \tag{1}$$

2.2. Safety and Feasibility Constraints

In forest fire extinguishing operations, besides optimizing the path, ensuring the safe flight of UAVs is equally crucial. In the forest environment, solid obstacles (e.g., trees, rocks) and dangerous zones (e.g., flames, thick smoke) collectively pose potential threats to UAV safety. For analysis, denote the set of all threat sources as K, and approximate each threat source as a cylinder with center C_k (coordinate) and radius R_k (as shown in Figure 1). For a path segment $|\overrightarrow{P_{ij}P_{i,j+1}}|$, its threat cost relates to the distance C_k from the segment to D_k.

Considering the UAV's diameter D and safe distance S (for collision avoidance), the threat cost K for the obstacle set P_{ij} between waypoints F_2 is formulated as:

$$\begin{cases} F_2(X_i) = \sum_{j=1}^{n-1} \sum_{k=1}^{k} T_k(\overrightarrow{P_{ij}P_{i,j+1}}), \\ T_k(\overrightarrow{P_{ij}P_{i,j+1}}) = \begin{cases} 0 & ,if\ D_k > S + D + R_k \\ (S + D + R_k) - D_k & ,if\ D + R_k < D_k \leq S + D + R_k \\ \infty & ,if\ D_k \leq D + R_k \end{cases} \end{cases} \tag{2}$$

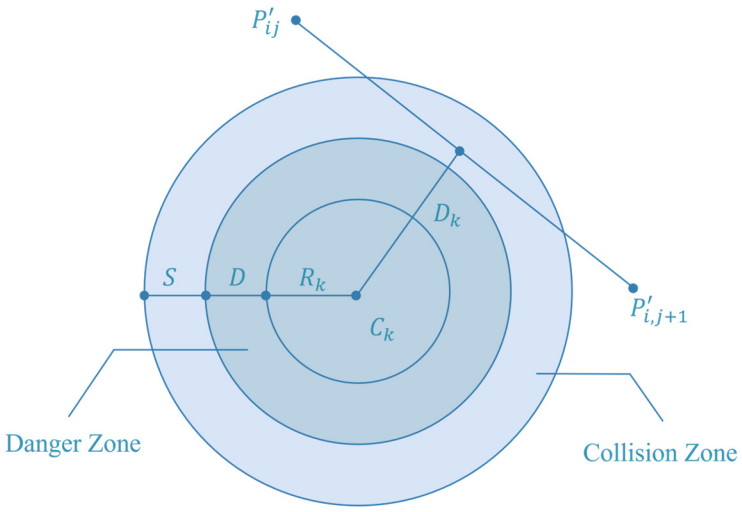


Figure 1. Determining Threat Costs.

Here, D is determined by the physical dimensions of the unmanned aerial vehicle (UAV), and the value of S is related to factors such as the complexity of the forest environment (for example, the density of trees affects the difficulty of flight obstacle avoidance) and the positioning accuracy (signal obstruction in the forest may affect the accuracy of GPS).

During forest firefighting operations, the flight altitude of UAVs also needs to be strictly restricted. On one hand, flying too low may expose the UAV to ground flames, hot gases, and other dangers; on the other hand, flying too high may exceed the effective monitoring and firefighting operation range. Let the minimum allowable flight altitude be $h_{\min}$ and the maximum flight altitude be $h_{\max}$.The altitude cost corresponding to the waypoint P_{ij} is calculated as follows:

$$H_{ij} = \begin{cases} \left| h_{ij} - \dfrac{(h_{\min} + h_{\max})}{2} \right|, & if\ h_{\min} \leq h_{ij} \leq h_{\max} \\ \infty & , otherwise \end{cases} \tag{3}$$

where h_{ij} is the actual flight altitude of the UAV relative to the ground at this waypoint, as shown in Figure 2. H_{ij} aims to keep the UAV's flight altitude within a reasonable range, avoiding it being too high or too low. Summing up H_{ij} for all waypoints, the altitude cost can be obtained:

$$F_3(X_i) = \sum_{j=1}^{n} H_{ij} \tag{4}$$

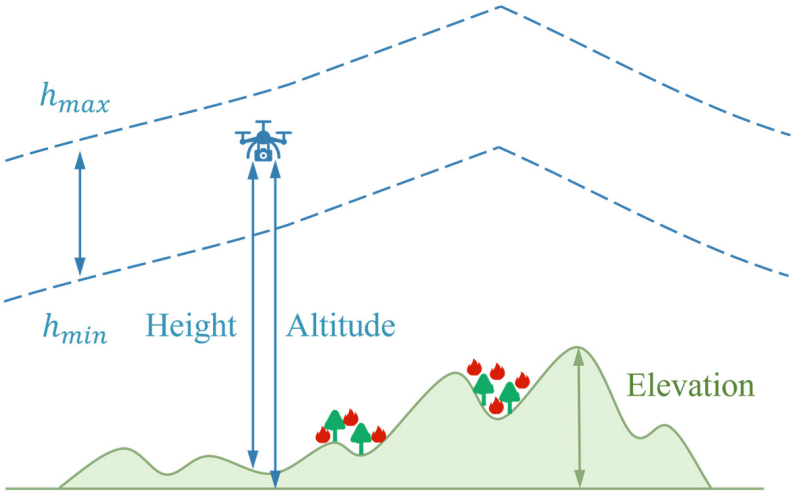


Figure 2. Illustration of altitude costs.

In addition, when flying in a complex forest environment, the turning and climbing operations of UAVs need to be reasonably controlled to ensure flight stability and safety. The smoothness cost is used to measure the rationality of the turning rate and climbing rate. As shown in Figure 3, the turning angle θ_{ij} is the included angle between the projections of two adjacent path segments on the horizontal plane Oxy, and it is calculated through the following steps: first, determine the unit vector $\vec{k}$ of the z-axis, and calculate the projection vectors.

$$\overrightarrow{P'_{ij}P'_{i,j+1}} = \vec{k} \times \left(\overrightarrow{P'_{ij}P'_{i,j+1}} \times \vec{k} \right) \tag{5}$$

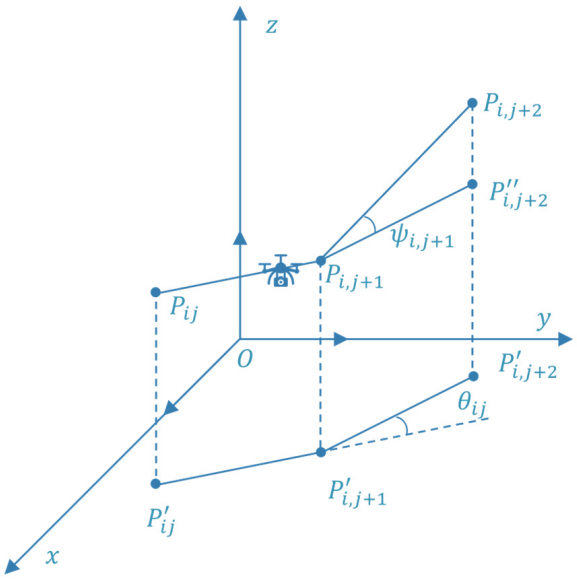


Figure 3. Calculation of turning and climbing angles.

Then, the turning angle is derived as:

$$\theta_{ij} = \arctan \frac{\left\| \overrightarrow{P'_{ij}P'_{i,j+1}} \times \overrightarrow{P'_{i,j+1}P'_{i,j+2}} \right\|}{\overrightarrow{P'_{ij}P'_{i,j+1}} \cdot \overrightarrow{P'_{i,j+1}P'_{i,j+2}}} \tag{6}$$

The climb angle ψ_{ij}, representing the angle between a path segment and its horizontal projection, is computed as:

$$\psi_{ij} = \arctan\left(\frac{z_{i,j+1} - z_{ij}}{\left\|\overrightarrow{P'_{ij}P'_{i,j+1}}\right\|}\right) \tag{7}$$

The smoothness cost function is defined as:

$$F_4(X_i) = a_1 \sum_{j=1}^{n-2} \theta_{ij} + a_2 \sum_{j=1}^{n-1} \left|\psi_{ij} - \psi_{i,j-1}\right| \tag{8}$$

where a_1 and a_2 are penalty weights for turning and climb angles, respectively. These coefficients regulate the rationality of turning and climbing maneuvers, preventing abrupt UAV movements.

2.3. Wind Field

In forest environments, complex terrain features (e.g., interlacing of valleys and peaks) cause significant variations in the intensity and direction of mountain winds, posing severe challenges to UAV flight. Mountain winds not only disrupt UAV flight stability but also increase energy consumption remarkably. Ignoring mountain winds during path planning may lead to UAV deviation from the planned route, reduced operational efficiency, and even failure to reach fire scenes in time. Taking the Ferno Islands forest area in Australia as an example, this study constructs a 3D environmental model using a digital elevation model (DEM) and simulates mountain winds via a segmented uniform wind field method. DEM elevation data accurately represent terrain features in the flight space, as shown in Figure 4, which includes top and three-dimensional view of the wind field, restricted flying zones, topographic map of the forest farm, and the comprehensive map.

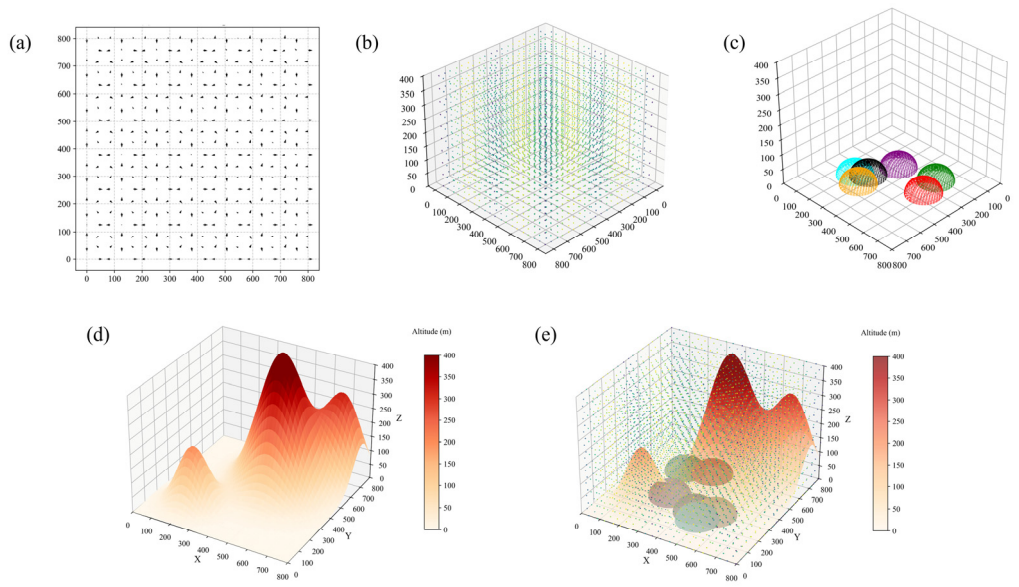


Figure 4. Environmental model ensemble, (**a**) Top view of wind field, (**b**) Three-dimensional view of wind field, (**c**) Restricted flying zones, (**d**) Topographic map of forest Farm, (**e**) Map of comprehensive environment.

To precisely simulate the impact of mountain winds on UAV path planning, a velocity-free variable wind field modeling method is proposed. Based on the assumption of a segmented uniform wind field, this method integrates displacement vector synthesis with spherical vector encoding. It abandons the traditional velocity synthesis model, using only distance ρ and angle (ψ, ϕ) parameters of spherical vectors, simplifying calculations and improving modeling efficiency.

Specifically, the planning area is divided into M sub-regions $\{(R_m)\}$. Within each sub-region, the horizontal wind field is a constant displacement vector $\vec{d}_{wind,m} = (d_{wind,u,m}, d_{wind,v,m})^T$ (unit: m, additional displacement during flight time). The horizontal planning displacement is determined by a spherical vector (ρ_k, ψ_k, ϕ_k). Superimposing the wind field's additional displacement with horizontal planning displacement gives the actual horizontal displacement $\vec{d}_{actual,h,k} = \vec{d}_{plan,h,k} + \vec{d}_{wind,m}$. The actual heading angle is $\phi_{actual,k} = \text{arctan2}(d_{actual,v,k}, d_{actual,u,k})$ with deviation $\Delta\phi_k = \phi_{actual,k} - \phi_k$ from the planning azimuth angle ϕ_k Accumulating absolute deviations of all path segments yields the heading deviation cost:

$$F_5 = \sum_{k=1}^{M} |\Delta\phi_k| \tag{9}$$

where i indexes the i-th object of interest, used to denote distinct instances for objective function evaluation.

2.4. Multi-Objective Problem Formulation and Fitness Function

In forest fire suppression missions, UAV path planning can be formulated as a multi-objective optimization problem (MOP) that seeks to simultaneously minimize five conflicting criteria. Mathematically, this problem is characterized as a vector minimization:

$$\begin{aligned} &min_{\mathbf{P}=\{P_0,P_1,\ldots,P_n\}} F(\mathbf{P}) = [F_1(\mathbf{P}),\ F_2(\mathbf{P}),\ F_3(\mathbf{P}),\ F_4(\mathbf{P}),\ F_5(\mathbf{P})]^T \\ &s.t. \quad \begin{cases} d(\overline{P_iP_{i+1}}, T_k) \geq R_k + S + D & \forall k \in \mathcal{T}, \forall i \in [0, n-1] \\ h_{min} \leq h_i \leq h_{max} & \forall i \in [0, n] \\ \psi_{i,i+1} \leq \psi_{max},\ \theta_{i,i+1} \leq \theta_{max} & \forall i \in [0, n-2] \\ |\Delta\phi_{i,i+1}| \leq \Delta\phi_{max} & \forall i \in [0, n-1] \end{cases} \end{aligned} \tag{10}$$

Here, $P = \{P_0, P_1, \ldots, P_n\}$, denotes the UAV path, with $P_i = (x_i, y_i, z_i)$ as 3D waypoints (P_0: start; P_n: target). The 5D objective vector $F(\mathbf{P})$ includes: F_1 (path length, sum of Euclidean distances), F_2 (threat cost, penalizing proximity to cylindrical zones $\mathcal{T}$, F_3 (altitude cost, penalizing deviations from $([h_{min}, h_{max}])$), F_4 (smoothness cost, penalizing excessive turning/climbing angles $(\psi_{i,i+1}, \theta_{i,i+1})$), F_5 (wind deviation cost, sum of heading deviations $\Delta\phi_{i,i+1}$).

Within the PVDE-MOJS meta-heuristic framework, the weighted aggregation presented in Equation (10) functions exclusively as a fitness function, enabling computationally efficient evaluation of solutions during iterative optimization. This scalar formulation facilitates rapid population screening; however, it neither alters the problem's fundamental multi-objective nature nor directly generates Pareto-optimal solutions. The formula for the fitness function is as follows:

$$F(X_i) = \sum_{k=1}^{5} b_k F_k(X_i) \tag{11}$$

where b_k are weight coefficients balancing constraint priorities (e.g., b_1 for path length in time-critical scenarios, b_2 for threat avoidance in obstacle-dense terrains). $F_1(X_i)$–$F_5(X_i)$ represent costs for path length (Equation (1), minimizing flight time), threats (Equation (2), obstacle/wildfire avoidance), smoothness (Equation (4), stable maneuvering), flight alti-

tude (Equation (8), safe operational range), and wind field (Equation (9), wind-induced trajectory deviation). Notably, the fitness function (Equation (10)) is non-convex for two reasons: the piecewise threat term (Equation (2)) induces discontinuous cost jumps at threat boundaries, violating convexity's smoothness with abrupt shifts; wind-induced nonlinearity (Equation (9)) yields quadratic terms via airspeed–wind interactions, producing a non-constant Hessian lacking global positive semi-definiteness—a defining convexity trait.

3. Parallel Vectorized Differential Evolution for Multi-Objective Jellyfish Search Algorithm

Based on the function in the previous section, the UAV path planning problem is transformed into an optimization problem of finding the optimal path that minimizes F. Since F is usually a complex multimodal function, using classical methods like the hill-climbing algorithm is infeasible due to local optimal solutions. Thus, heuristic and meta-heuristic methods are often adopted to obtain high-quality solutions within a reasonable time.

The improved multi-objective jellyfish search algorithm proposed in this paper, as a novel and efficient meta-heuristic optimization technique, is suitable for solving such complex problems. This section will introduce the improved multi-objective jellyfish search algorithm, which incorporates unique strategies to enhance search efficiency. Its superiority over the original Multi-Objective Jellyfish Search (MOJS) has been verified via U1–U9 test functions, and the content related to the basic process is as follows.

3.1. Improved Multi-Objective Jellyfish Search Algorithm

In practical applications, traditional optimization algorithms have significant bottlenecks. Some algorithms rely on fixed-direction vector update mechanisms, which easily lead to frequent UAV path backtracking in complex scenarios such as dense forest obstacles or dynamic fire areas, resulting in low search efficiency. The demand for path evaluation at the million-time scale brought by high-dimensional decision variables drastically increases the time consumption of traditional serial computing modes. Heuristic algorithms, due to their characteristics of not relying on precise mathematical models, emphasizing global search and adaptive adjustment, have become an effective means to break through bottlenecks. Among them, meta-heuristic algorithms, classic population-based heuristic methods, show great potential in the field of multi-objective optimization, and their population optimization paradigm has become the mainstream direction for solving complex problems.

The parallel vectorized differential evolution Multi-Objective Jellyfish Search algorithm (PVDE-MOJS) proposed in this paper is based on inheriting the advantages of heuristic algorithms, specifically improving the path planning efficiency and solution quality in high-dimensional scenarios and providing a new technical path for multi-objective optimization in complex environments.

The Multi-Objective Jellyfish Search algorithm (MOJS) was proposed by Jui-Sheng Chou et al. in 2020 [37]. Its inherent advantages endow it with unique value when dealing with such complex scenarios. This algorithm is based on the bionic principle of jellyfish swarm movement. By switching between the global exploration of "following ocean currents" and the local exploitation of "intra-swarm movement", it can naturally balance the breadth and depth of solution space search. Compared with traditional algorithms such as particle swarm optimization, MOJS only requires two internal parameters, has a concise encoding, and is easy to implement. Especially in high-dimensional spaces, it can avoid premature convergence through a time control mechanism, making it more suitable for the collaborative optimization of multiple objectives (such as shortest path, obstacle-avoidance safety, and optimal endurance) in forest environments.

Compared with the original MOJS, the improved Parallel Vectorized Differential Evolution Multi-Objective Jellyfish Search algorithm (PVDE-MOJS) breaks through traditional limitations with three core technical innovations, providing a more efficient solution for multi-objective optimization in complex high-dimensional scenarios. The algorithm innovatively introduces a parallel evaluation system, leveraging multi-process pools and dynamic load-balancing techniques to significantly enhance the path evaluation efficiency in high-dimensional environments. The vectorized screening strategy, combined with matrix operations and Numba parallel technology, greatly accelerates the screening speed of the Pareto-front solution set. The intelligent mutation search mechanism adjusts differential evolution operators through a dynamic decay factor, achieving a precise balance between global exploration and local exploitation, effectively avoiding the "blind roaming" problem in path planning.

In simulated forest environment tests, the PVDE-MOJS algorithm can generate high-quality solution sets that balance multiple objectives such as shortest path, obstacle avoidance safety, and optimal energy consumption in a short time. This optimization strategy, which deeply integrates bionic principles with cutting-edge computing technologies, not only inherits the global search advantages of the original algorithm but also achieves significant improvements in adaptability in complex environments, computational efficiency, and solution accuracy. It provides reliable technical support for UAV path planning in emergency scenarios such as forest fire rescue and material delivery.

3.2. Algorithm Framework

The flowchart of the entire PVDE-MOJS algorithm used for the model solution is shown in Figure 5, and the steps for the model solution are as follows.

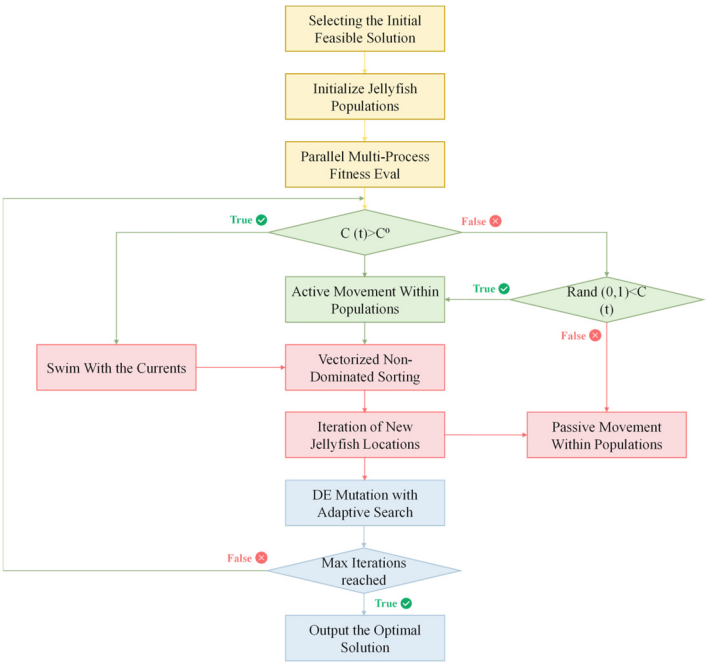


Figure 5. The flowchart of PVDE-MOJS Algorithm (Note: C^0 represents the initial current intensity threshold, where $C^0 = C(0) = 1$ corresponding to Equation (16) at t = 0 (i.e., the initial iteration), "$Rand(0,1) < C(t)$" (jellyfish perform active movement if random [0, 1] value < dynamic $C(t)$, else passive movement).

3.2.1. Population Initialization

In the PVDE-MOJS algorithm framework, constructing the initial population of jellyfish is a crucial starting point. Since jellyfish follow a normal distribution in all dimensions,

the algorithm is prone to the risk of premature convergence. Based on this, PVDE-MOJS introduces the logistic chaotic mapping method. Through this method, N_{pop} data strings J are generated. Each data string J_i corresponds to an individual jellyfish, and the data sets of these individual jellyfish form the required initial population.

$$J_i = (x_1, x_2 \cdots x_i)^T_{1\times v} \tag{12}$$

$$J_0 = J_{\min} + rand(J_{\max} - J_{\min}) \tag{13}$$

$$J_{i+1} = \eta J_i(1 - J_i), 0 \le J_0 \le 1 \tag{14}$$

Equation (11) defines the representation form of an individual jellyfish in a multi-dimensional space, Here, J_i is the decision vector of the i-th jellyfish, locating it in a v-dimensional space. $x_1, x_2 \cdots x_i$ are its components, each a distinct variable influencing multi-objective outcomes. Equation (12) is used to determine the starting value of the initial jellyfish population position, where $J_{\max}$ and $J_{\min}$ are the maximum and minimum values of the search space in the corresponding dimension, respectively. $J_{\max}$ and $J_{\min}$ are static boundary parameters preset during the algorithm initialization phase, used to limit the value range of the initial position J_0 of jellyfish individuals. The rand function here follows a uniform distribution over [0, 1), generating a random scalar that, when scaled by $J_{\max} - J_{\min}$ and shifted by $J_{\min}$, ensures J_0 is uniformly distributed within $[J_{\min}, J_{\max})$. Equation (13) describes the iterative process from one chaotic value to the next, enriching the diversity of population initialization. Meanwhile, to avoid falling into some special values that may lead to undesirable behavior of the algorithm, $J_0 \notin \{0, 0.25, 0.75, 1\}$.

In this work, the population size is configured as 100. Each individual within the initial population undergoes sequential verification of constraint conditions. Random individuals that violate the model's constraints are removed through a fitness penalty mechanism: the greater the number of constraints an individual violates, the higher the likelihood of its elimination. Eliminated individuals are substituted with new initial individuals until the population size aligns with the predefined requirement.

3.2.2. Optimization of Fitness Evaluation Based on Multi-Core Parallelism

When the MOJS algorithm deals with high-dimensional multi-objective optimization problems, fitness evaluation often becomes a computational bottleneck. To address this, the PVDE-MOJS algorithm proposes an acceleration strategy based on task parallelism: it relies on a multi-process pool to achieve parallel distribution of individual evaluation tasks. By dynamically dividing the population size N into multiple batches, each logical core processes multiple batches of tasks simultaneously to maximize the utilization of hardware resources. Here, C denotes the number of logical cores of the CPU. The number of individuals processed in each batch is determined by Formula (15):

$$N' = \frac{N}{(C \times F)} \tag{15}$$

$$\begin{cases} Delay_{t+1} = Delay_t \times (1 + \alpha), if\ \rho(t) > \theta \\ Delay_{t+1} = Delay_t \times (1 - \beta), if\ \rho(t) \le \theta \end{cases} \tag{16}$$

Formula (16) is key to dynamic load balancing. The system monitors real-time CPU utilization $\rho(t)$: if overloaded ($\rho(t) > \theta$), it increases scheduling delay via $Delay_{t+1} = Delay_t \times (1 + \alpha)$; if underloaded ($\rho(t) \le \theta$), it reduces delay through $Delay_{t+1} = Delay_t \times (1 - \beta)$. This keeps the system optimally loaded during iteration.

Here F is the hyper-threading factor. Meanwhile, a dynamic load balancing mechanism based on CPU utilization is introduced: it monitors the system in real time and compares it

with the threshold $\rho(t)$. When $\rho(t) > \theta$, the scheduling delay is increased, otherwise, it is decreased. The coefficients α and β are used to dynamically balance the system load. This strategy optimizes the time complexity of large-scale population iterations to a linear level through task batching and dynamic resource scheduling, effectively breaking through the evaluation efficiency bottleneck in high-dimensional scenarios and meeting the real-time optimization requirements of dynamic environments.

This parallel fitness evaluation is not an isolated module but structurally integrated with the vectorized non-dominated sorting module (as elaborated in Section 3.2.5). The fitness data generated in parallel is directly streamed into vectorized matrix operations for dominance assessment and crowding distance computation, forming an integrated pipeline that eliminates redundant data transfer and accelerates solution set updates—a critical distinction from the serial workflow of the original MOJS. For 3D path planning involving 12 waypoints (36-dimensional decision space) in forest scenarios, this parallel strategy, when combined with vectorized operations, reduces total computation time by approximately 60% compared to the serial MOJS (validated on a 16-core CPU). More importantly, the dynamic load balancing mechanism (Equations (15) and (16)) prioritizes fitness evaluation for individuals near high-threat zones and wind field boundaries, directing computational resources toward critical search regions. This targeted optimization directly enhances the algorithm's capacity to explore Pareto-optimal paths in complex terrain, rather than merely accelerating computation. Unlike the original MOJS, which executes fitness evaluation and non-dominated sorting sequentially, PVDE-MOJS's parallelized pipeline enables the screening of high-quality solutions (Pareto frontier candidates) to proceed in parallel with the evaluation of the next generation. This synergy efficiently utilizes iteration budgets to expand solution diversity.

3.2.3. Behavior Regulation Mechanism of PVDE-MOJS Algorithm

In the PVDE-MOJS algorithm, the behavior regulation mechanism simulates the foraging behavior of jellyfish in natural ocean environments and optimizes search efficiency through the synergy of time control and boundary constraints:

$$C(t) = \left| \left(1 - \frac{t}{t_{\max}}\right)(2 \times rand(0,1) - 1) \right| \tag{17}$$

$$\begin{cases} J'_{i,d} = (J_{i,d} - U_{i,d}) + J_{\min}(d), if\ J_{i,d} > J_{\max,d} \\ J'_{i,d} = (J_{i,d} - L_{i,d}) + J_{\max}(d), if\ J_{i,d} < J_{\min,d} \end{cases} \tag{18}$$

The time control mechanism is designed based on the dual behavior modes of jellyfish: ocean current following (global exploration) and intra-population movement (local exploitation). Using Equation (16), where t denotes the current iteration and $t_{\max}$ the maximum iteration, it dynamically switches modes: when $Rand(0,1) < C(t)$, jellyfish trigger the "ocean current following" mode, expanding the search range through global movement; when $Rand(0,1) \geq C(t)$, they switch to "intra-population movement" for local fine-grained search. Here, $C(t)$ originates from an initial current intensity threshold $C^0 = 1$ (i.e., $C(0) = C^0$) at the first iteration, $t = 0$, which gradually decays to 0 as t approaches $t_{\max}$. This initial value C^0 ensures that "ocean current following" (global exploration) is prioritized in the early stages of optimization, enabling the population to broadly traverse the search space before shifting toward local exploitation. This couples iteration count with random factors, balancing global exploration and local exploitation.

The boundary constraint mechanism addresses jellyfish boundary crossing via Earth's spherical topology. For the i-th jellyfish in dimension d, $J_{i,d}$ is its fitness; $J_{\min,d}$ and $J_{\max,d}$ define the search range's lower and upper thresholds, with $U_{i,d} = J_{i,d} - J_{\max,d}$ (excess over

the upper threshold) and $L_{i,d} = J_{\min,d} - J_{i,d}$ (deficit below the lower threshold) measuring boundary violations. $J_{\min}(d)$ and $J_{\max}(d)$ are preset valid boundaries after adjustment. As Equation (17) shows, if $(J_{i,d} > J_{\max,d})$, $(J'_{i,d} = (J_{i,d} - U_{i,d}) + J_{\min}(d))$ maps it to the lower valid boundary; if $(J_{i,d} < J_{\min,d})$, $(J'_{i,d} = (J_{i,d} - L_{i,d}) + J_{\max}(d))$ rebounds it to the upper valid boundary, ensuring continuous exploration within the spherical structure and avoiding invalid out of range computations.

By synergizing behavior mode switching (time control) and spatial constraints (boundary regulation), the algorithm achieves a balance between search breadth and precision in complex optimization scenarios, significantly enhancing solution efficiency for high-dimensional problems.

3.2.4. Optimization and Intelligent Enhanced Search Phase

The search process of the PVDE-MOJS algorithm achieves efficient optimization via a series of ingeniously designed mechanisms, encompassing two primary aspects: optimization search and intelligent enhanced search.

In the optimization search mechanism, three search behaviors are defined. During following current search, the influence of food-rich ocean currents on jellyfish is fully leveraged. The ocean current direction characterizes the average difference between the optimal jellyfish position and the population's jellyfish positions. Based on the assumption that jellyfish positions follow a normal distribution, which provides a probabilistic framework to model the randomness and regularity of their movement, the new position update formula is derived as follows. The normal distribution serves as a foundational premise for the initial model setup, and for the sake of focusing on the core algorithmic mechanism, it will not be elaborated in depth in the following sections.

$$J_i^{(t+1)} = J_i^{(t)} + u \cdot \vec{D},\ u \sim U(0,1) \tag{19}$$

$$\vec{D} = J^* - \beta \cdot u \cdot \left(e_C \cdot \frac{\sum_{i=1}^{N_{pop}} J_i}{N_{pop}} \right), u \sim U(0,1) \tag{20}$$

where β = 3 (distribution coefficient, scalar), J^* (current optimal position, vector), N_{pop} (population size, scalar), and e_C (attraction control factor, scalar). From a bionic perspective, the algorithm's convergence trend mimics the natural behavior where jellyfish are driven by ocean currents. Specifically, the ocean current direction, quantified by the average vector $\vec{E}$ representing the collective deviation of all jellyfish positions from the current optimal position J^*, guides the movement pattern, making the population gradually converge toward more promising regions in the search space.

$$\vec{E} = \frac{1}{N_{pop}} \cdot \sum_{i=1}^{N_{pop}} \vec{E}i \tag{21}$$

In Formula (19), the summation index i ranges from 1 to N, where N denotes the total number of individuals in the population; in Formula (20), the summation index d ranges from 1 to D, with D representing the dimensionality of the problem.

$$\vec{E}i = J^* - e_C J_i \tag{22}$$

$$Set\ df = e_c\ \cdot \mu df \tag{23}$$

$$\overrightarrow{E} = J^* - e_C \frac{\sum J_i}{N_{pop}} = J^* - e_C \mu \tag{24}$$

In Formula (21), $\overrightarrow{E}_i$ is a vector representing the position deviation of the i-th individual from the optimal position, adjusted by the attraction control factor e_C. To streamline the subsequent derivation of population-level behavioral trends, we introduce auxiliary variables in Formula (22): let df represent a transient scalar quantity (employed to facilitate the aggregation of individual-level dynamics into population-level patterns) and μdf denote a coefficient that encapsulates the statistical characteristics of the population's spatial distribution. In Formula (22), μ serves as an average coefficient that captures the macroscopic distribution features of the population's positions, thereby embodying the collective movement bias of the population within the search space. The vector $\overrightarrow{Tr}$, in turn, provides a quantitative descriptor of the dominant directional tendency guiding the population's evolutionary dynamics.

Population movement includes two modes: passive movement and active movement. Determined by Rand$(0,1)$ and dynamic $C(t)$ (Equation (16)), in the early stage, $C(t) \approx 1$, so $Rand(0,1) < C(t)$ (high probability) triggers active movement (Equations (25) and (26), moving toward food-rich individuals); in the later stage, $C(t) \approx 0$, so Rand$(0,1) \geq C(t)$ (high probability) triggers passive movement (Equation (24), $(\tau = 0.1)$). The movement formulas are as follows:

$$J_i^{(t+1)} = J_i^{(t)} + \tau \times rand(0,1)(J_{\max} - J_{\min}) \tag{25}$$

$$J_i^{(t+1)} = J_i^{(t)} + rand(0,1) \times \overrightarrow{Op} \tag{26}$$

$$\overrightarrow{Op} = \begin{cases} J_j^{(t)} - J_i^{(t)}, if\ f\left(J_i^{(t)}\right) > f\left(J_j^{(t)}\right) \\ J_i^{(t)} - J_j^{(t)}, if\ f\left(J_j^{(t)}\right) > f\left(J_i^{(t)}\right) \\ 0 \quad , if\ f\left(J_j^{(t)}\right) = f\left(J_i^{(t)}\right) \end{cases} \tag{27}$$

To further enhance the local search capability of the algorithm, an intelligent search mechanism, namely the differential evolution (DE) mutation operator [38], is introduced in the intra-population movement stage. On one hand, a dynamic decay factor F is used to balance global exploration and local exploitation. As the number of iterations increases, the factor F linearly decays from 0.9 to 0.1. On the other hand, the difference vectors of random individuals in the population are used to construct non-linear search directions, which refer to direction adjustments integrating population distribution characteristics and adaptive coefficients, and then mutant individuals are generated. The specific formula is as follows:

$$J_{\text{mut}}^{(t)} = J_a^{(t)} + F \times (J_b^{(t)} - J_c^{(t)}) \tag{28}$$

where J_a, J_b, and J_c are randomly selected non-current individuals, whose diversity reflects population distribution (supporting exploration of heterogeneous solution spaces). The scaling factor F (an adaptive coefficient, dynamically tuned to balance exploration–exploitation) introduces non-linearity, the differential term $(J_b^{(t)} - J_c^{(t)})$ captures population-driven directional information, and scaling it with F before combining with $J_a^{(t)}$ creates search directions deviating from linear gradients. This aligns with differential evolution principles, supporting the algorithm's balance of global exploration and local exploitation and avoiding "blind roaming" in path planning.

Subsequently, crossover operations construct trial vectors $U_i^{(t)}$ via binomial recombination:

$$U_{i,j}^{(t)} = \begin{cases} J_{\text{mut},j}^{(t)} & \text{if rand}(0,1) \leq CR \vee j = j_{\text{rand}} \\ J_{i,j}^{(t)} & \text{otherwise} \end{cases} \tag{29}$$

Here, the crossover rate $CR = 0.9$ controls dimension-wise inheritance from the donor vector $J_{\text{mut},j}^{(t)}$, while j_{rand} ensures at least one mutated parameter is retained. Selection operations then apply Pareto-crowding criteria:

$$J_i^{(t+1)} = \begin{cases} U_i^{(t)} & \text{if } U_i^{(t)} \prec J_i^{(t)} \\ U_i^{(t)} & \text{if } U_i^{(t)} \not\succ J_i^{(t)} \wedge \mathcal{H}(U_i^{(t)}) > \mathcal{H}(J_i^{(t)}) \\ J_i^{(t)} & \text{otherwise} \end{cases} \tag{30}$$

Here, t is the iteration (or evolution) step, marking the optimization phase timing; i is the index of an individual in the population to distinguish candidate solutions; $J_i^{(t)}$ is the current solution of the i-th individual at iteration t, a candidate entity to be updated or retained; $U_i^{(t)}$ is the trial solution for the i-th individual at iteration t, generated via crossover/mutation for competitive replacement; $\prec$ denotes multi-objective dominance, $\not\succ$ is non-dominance, indicating solutions have trade-offs across objectives; $\mathcal{H}(\,)$ is defined for Pareto-optimal solutions $P = \{x_1, \ldots, x_k\}$ by sorting P by each objective $f_m(m = 1, \ldots, M)$ to get $x_{(1)}, \ldots, x_{(k)}$ setting $H_m(x_{(1)}) = H_m(x_{(k)}) = +\infty$ for boundaries and $H_m(x(\mathrm{i})) = f_m(x(\mathrm{i}+1)) - f_m(x(\mathrm{i}-1))$ for $2 \leq i \leq k-1$ total $H(x_i) = \sum_{m=1}^{M} H_m(x_i)$ with larger values indicating sparser Pareto front regions. It is optimized via vectorized matrix operations and Numba parallel technology (Section 3.2.5).

3.2.5. Multi-Objective Vector Optimization Unit

In the MOJS algorithm, non-dominated sorting and crowding degree calculation often rely on nested loops for pairwise individual comparison, leading to high time complexity and low efficiency in processing large-scale solution sets. To enhance the vector optimization efficiency of the MOJS algorithm, the PVDE-MOJS algorithm incorporates an optimization scheme for non-dominated sorting and crowding degree calculation based on matrix operations [39]. After each position iteration of the algorithm, the multi-objective vector optimization unit achieves efficient screening of the current solution set through the synergy of a vectorized non-dominated sorting algorithm and Numba parallel technology, so as to determine high-quality iterative generations. In the multi-objective value processing step, this unit first converts multi-objective parameters (such as construction cost, response time, and radiance) of population individuals into a matrix F and further constructs a dominance matrix to judge the dominance relationship among individuals quickly. This matrix-based processing mechanism transforms traditional nested loop judgments into efficient matrix operations. Combined with Numba just-in-time compilation technology, the crowding degree d_i is calculated, where d_i^k is the crowding degree of the individual target x_i. While maintaining the diversity of the solution set, it significantly improves the screening efficiency. By accurately evaluating the distribution density of individuals, it ensures the uniform distribution of the solution set on the Pareto front, thus optimizing the scientific nature of the algorithm's search process.

$$F \in R^{N \times M} \tag{31}$$

$$\mathbf{D}_{ij} = \begin{cases} 1, if \forall k \in \{1, \ldots, M\} : f_k(x_i) \leq f_k(x_j) \wedge \exists K' \in \{1, \ldots, M\} : f_k(x_i) < f_k(x_j) \\ 0, \qquad \text{other wise} \end{cases} \tag{32}$$

$$d_i = \sum_{k=1}^{M} d_i^k \tag{33}$$

After the above two-stage processing, the multi-objective vector optimization unit can efficiently screen out Pareto front individuals. These individuals with excellent performance in multi-objective dimensions will serve as high-quality generations for the next iteration, guiding the algorithm to continuously approach the optimal solution and promoting the iterative optimization of the search process.

3.2.6. Position Iteration Optimization

In the PVDE-MOJS algorithm, the position iteration optimization stage achieves directed exploration of the search space via quantitative evaluation of individual fitness. A multi-objective weighted evaluation function scores each individual's quality:

$$\min fit = \lambda \cdot Z + \theta \tag{34}$$

where $\lambda = (1\ , 1\ , -1)$ (weight vector for objective balance), Z (multi-objective matrix: construction cost, response time, radiance):

$$Z = \begin{Bmatrix} A \cdot J \\ B \cdot J \\ Z_3 \end{Bmatrix}_{3 \times 1} \tag{35}$$

where $A = (A_1 \cdots A_l\ )_{1 \times v}$, $B = (B_1 \cdots B_l\ )_{1 \times v}$: feature matrices; J: individual position vector. A penalty mechanism (Equation (33)) handles constraint violations (dynamic p based on violation degree).

After the fitness evaluation, the optimal position is stored as J^*. Postiteration, check termination: if not met, update $C(t)$ and continue; else, output optimal position and resource allocation.

$$\theta = pL \tag{36}$$

3.3. Simulation Results

To validate the performance enhancement of PVDE-MOJS, this study conducts comparative experiments on the standard test suite UF1–UF10 and systematically analyzes it against the original MOJS algorithm. The experimental setup involves a fixed population size N = 100, a maximum iteration count $T_{\max}$ = 50, and an initial population uniformly generated within specified domains (U1: $[-5, 5]$, others: [0, 1]), with solutions strictly constrained to the feasible space. The inverted generational distance (IGD, Equation (34)) is used as the core evaluation metric, which measures both convergence and distribution (lower values indicate better performance):

$$IGD = \frac{\sum_{v \in P^*} min_{u \in P} d(v, u)}{|P^*|} \tag{37}$$

Here, P^* represents the true Pareto front samples, P is the output of the algorithm, and $d(v, u)$ denotes the Euclidean distance.

Simulations (30 runs per function, Figures 6–10) show that PVDE-MOJS outperforms MOJS. Its solutions (red) closely match the true Pareto front (TPF, blue) in UF1–UF4,

with improved TPF surface approximation in 3D problems (UF8–UF10), reducing the average distance to the ideal front. In UF2–UF3, red solutions fill green gaps of MOJS, achieving uniform TPF distribution. For multi-modal (UF6) and high-dimensional (UF9) problems, PVDE-MOJS covers TPF better (e.g., UF5 global coverage vs. MOJS local clustering, avoiding local optima). In conclusion, PVDE-MOJS excels in convergence, diversity, robustness, boundary avoidance, and TPF fitting, demonstrating superiority in multi-objective optimization.

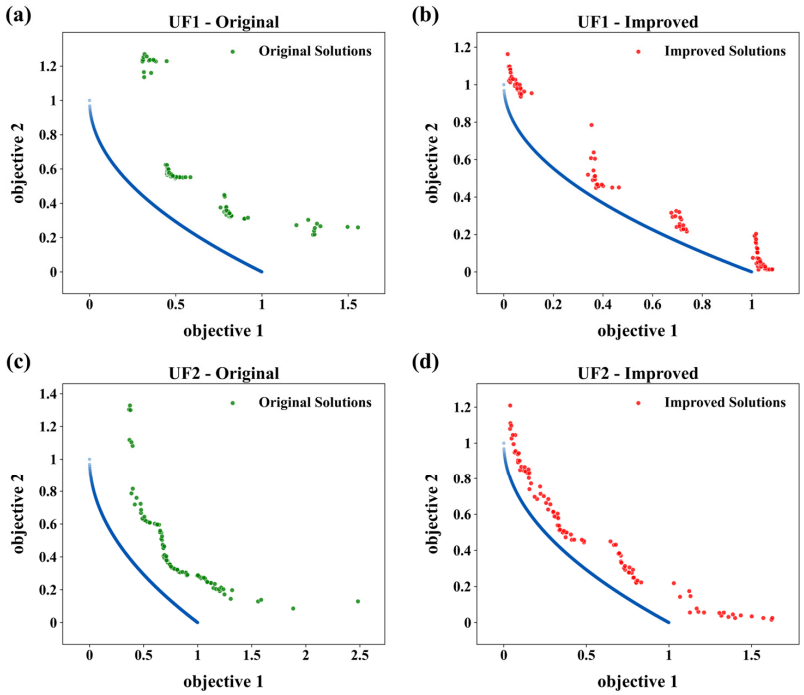


Figure 6. Comparison of solution distribution and Pareto frontiers, (**a**) UF1 with original algorithm; (**b**) UF1 with improved algorithm; (**c**) UF2 with original algorithm; (**d**) UF2 with improved algorithm.

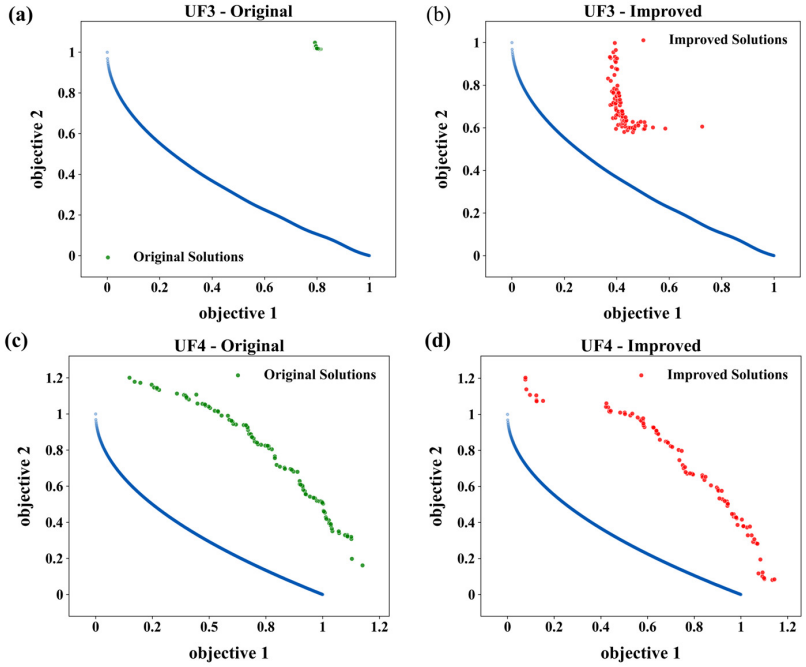


Figure 7. Comparison of solution distribution and Pareto frontiers, (**a**) UF3 with original algorithm; (**b**) UF3 with improved algorithm; (**c**) UF4 with original algorithm; (**d**) UF4 with improved algorithm.

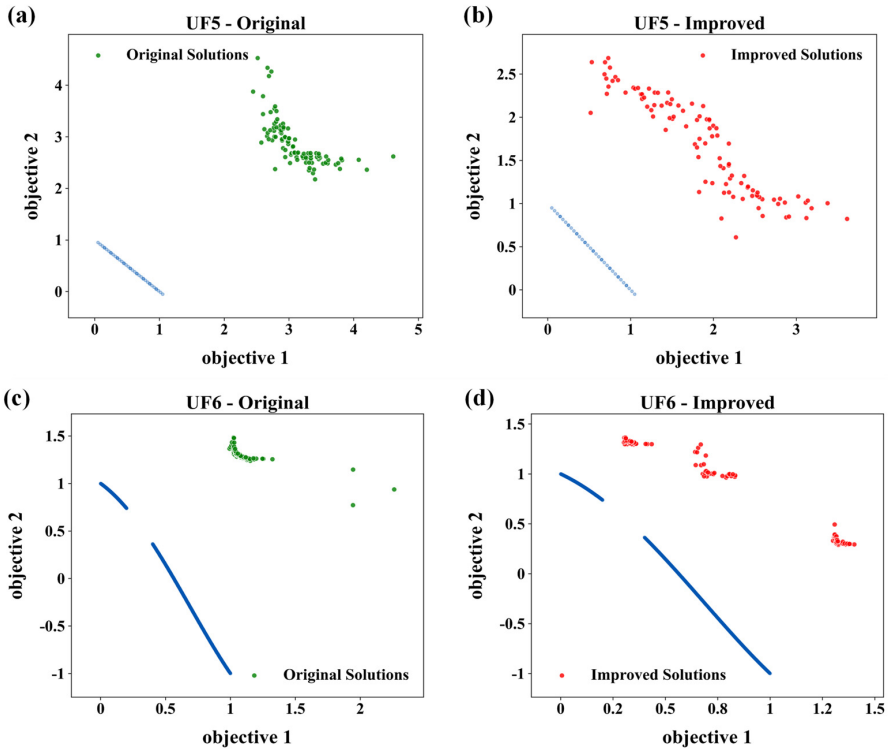


Figure 8. Comparison of solution distribution and Pareto frontiers, (**a**) UF5 with original algorithm; (**b**) UF5 with improved algorithm; (**c**) UF6 with original algorithm; (**d**) UF6 with improved algorithm.

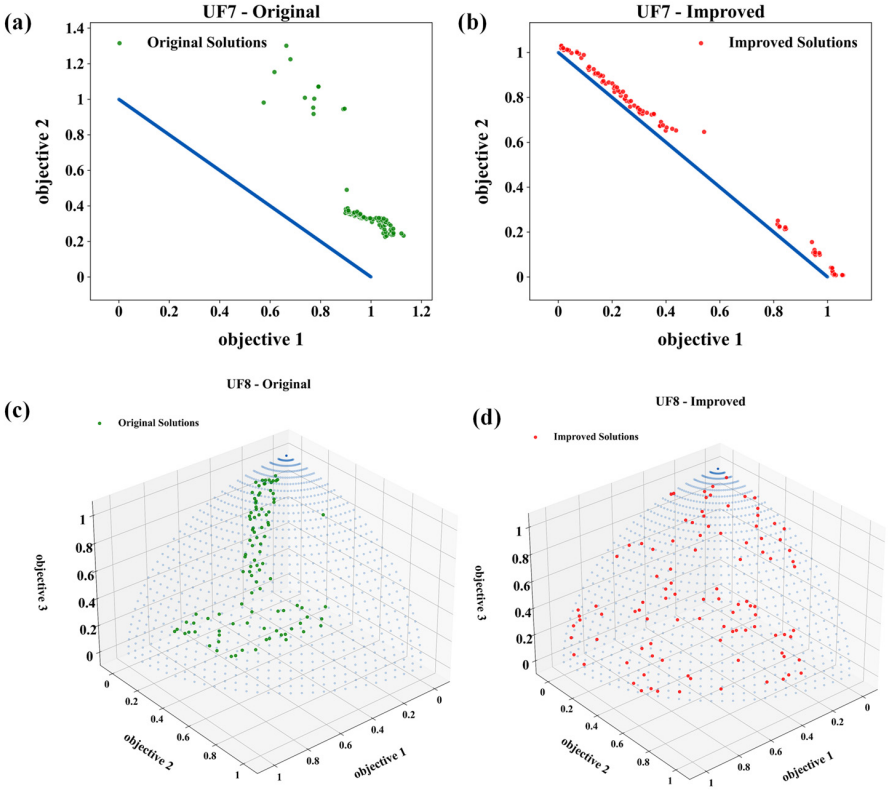


Figure 9. Comparison of solution distribution and Pareto frontiers, (**a**) UF7 with original algorithm; (**b**) UF7 with improved algorithm; (**c**) UF8 with original algorithm; (**d**) UF8 with improved algorithm.

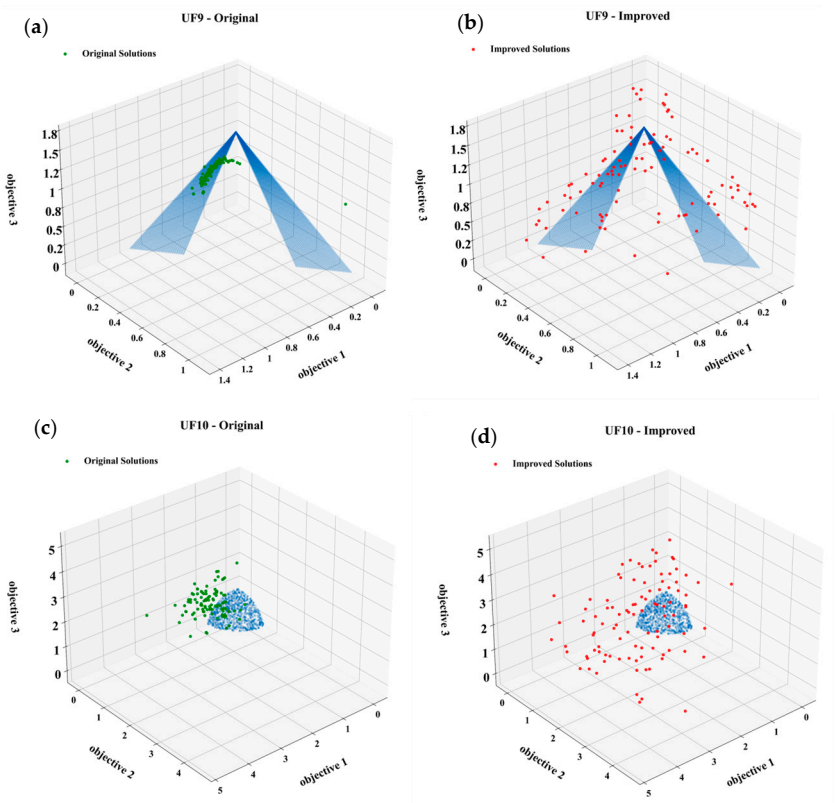


Figure 10. Comparison of solution distribution and Pareto frontiers, (**a**) UF9 with original algorithm; (**b**) UF9 with improved algorithm; (**c**) UF10 with original algorithm; (**d**) UF10 with improved algorithm.

4. PVDE-MOJS Algorithm for UAV Path Planning

Leveraging the maneuvering characteristics of UAVs, this paper proposes the PVDE-MOJS algorithm and provides its implementation to address the path planning problem.

4.1. PVDE-MOJS Path Planning Method Based on Spherical Vectors

PVDE-MOJS represents each flight path as a vector, describing the UAV's movement from one waypoint to another. In spherical coordinates, each vector has three components: polar angle $\rho\epsilon\left(0, path_{length}\right)$, elevation angle $\psi \in \left(-\pi/2, \frac{\pi}{2}\right)$, and azimuth angle $\varphi \in (-\pi, \pi)$. A 3N-dimensional supersphere represents N waypoints [40]:

$$\Omega_i : \Omega_i = (\rho_{i1}, \psi_{i1}, \phi_{i1}, \rho_{i2}, \psi_{i2}, \phi_{i2}, \ldots, \rho_{iN}, \psi_{iN}, \phi_{iN}), N = n - 2 \tag{38}$$

Velocity increments are described by:

$$\Delta\Omega_i = (\Delta\rho_{i1}, \Delta\psi_{i1}, \Delta\phi_{i1}, \Delta\rho_{i2}, \Delta\psi_{i2}, \Delta\phi_{i2}, \ldots, \Delta\rho_{iN}, \Delta\psi_{iN}, \Delta\phi_{iN}) \tag{39}$$

Define spherical vector $u_{ij} = \left(\rho_{ij}, \psi_{ij}, \varphi_{ij}\right)$ and velocity $\Delta u_{ij} = \left(\Delta\rho_{ij}, \Delta\psi_{ij}, \Delta\varphi_{ij}\right)$. The update equation for PVDE-MOJS is:

$$\Delta u_{ij}^{k+1} \leftarrow w^k \Delta u_{ij}^k + \eta_1 r_{1j}\left(q_{ij}^k - u_{ij}^k\right) + \eta_2 r_{2j}\left(q_{gj}^k - u_{ij}^k\right) \tag{40}$$

$$u_{ij}^{k+1} \leftarrow u_{ij}^k + \Delta u_{ij}^{k+1}, (i = 1, 2, \ldots, M; j = 1, 2, \ldots, N) \tag{41}$$

where $Q_i = (q_n, q_n, \cdots, q_{nN})$ (local optimal) and $Q_g = \left(q_{g1}, q_{g2}, \cdots, q_{gN}\right)$ (global optimal) are position sets. Waypoint projection to Cartesian coordinates:

$$\chi_{ij} = x_{i,j-1} + \rho_{ij} sin\psi_{ij} cos\phi_{ij} \tag{42}$$

$$y_{ij} = y_{i,j-1} + \rho_{ij} sin\psi_{ij} sin\phi_{ij} \quad (43)$$

$$z_{ij} = z_{i,j-1} + \rho_{ij} cos\psi_{ij} \quad (44)$$

The spherical vector method leverages polar, elevation, and azimuth components to link UAV velocity, turn, and climb angles, enhancing navigation safety. Unlike Cartesian space, it searches in spherical topology, improving optimal solution discovery and reducing search space via angle constraints (e.g., fixed-speed flight narrows ρ, boosting local search).

4.2. Simulation Setup

The evaluation scenario is constructed using a real digital elevation model (DEM) map acquired via LiDAR sensor, based on the 3D environmental model of the Philno Archipelago Forest Farm in Australia. Cylindrical threat regions (radii, 3D central coordinates) are listed in Table 1 (dangerous zones for UAV avoidance). UAV start/end positions (path planning) are in Table 2. Across three runs, UAV-specific start/end coordinates and path node counts are detailed in Table 3. The starting points, endpoints, and path nodes in this simulation strictly correspond to the definitions in Section 2: Tables 2 and 3 list the 3D coordinates of P_0 (starting points) and P_n (endpoints) for each UAV, while the 12 path nodes (excluding the start point) correspond to the intermediate waypoints P_0 to P_{12} in the sequence $P = \{P_0, P_1, \ldots, P_{12}\}$, ensuring consistency with the problem formulation.

Table 1. Threat Area Information.

Threat Area Number	Radius (R)	x-Coordinate	y-Coordinate	z-Coordinate
1	60	350	500	100
2	70	600	200	150
3	80	500	350	150
4	70	350	200	150
5	70	700	550	150

Table 2. UAV's Starting and Ending Point Location Information.

Location Type	x-Coordinate	y-Coordinate
Starting Point	400	100
Ending Point	900	550
Location Type	x-coordinate	y-coordinate

Table 3. Flight Data of UAVs under Different Operation Times.

Operation Times	UAV ID	Starting Position (x_0, y_0, z_0)	Ending Position (x_n, y_n, z_n)	Path Node Setting (Excluding Start Point)
First Run	1	150; 200; 150	800; 800; 150	12
First Run	2	400; 100; 150	900; 550; 150	12
Second Run	1	123; 222; 152	811; 874; 157	12
Second Run	2	477; 117; 159	957; 557; 152	12
Third Run	1	118; 274; 112	511; 804; 145	12
Third Run	2	450; 127; 157	939; 547; 149	12

4.3. Path Planning Results and Analysis

Figure 11 displays the path planning results for multiple UAVs. It can be seen from the figure that, during the flight, the UAVs successfully avoided obstacles such as trees and rocks in the forest farm and did not enter restricted flight areas like the preset no-fly zones. This result fully verifies the effectiveness of the proposed algorithm in path planning,

demonstrating its excellent ability to avoid obstacles and restricted areas. It shows that the algorithm has strong potential for practical application and can realize safe and reliable UAV path planning in complex environments such as forest firefighting.

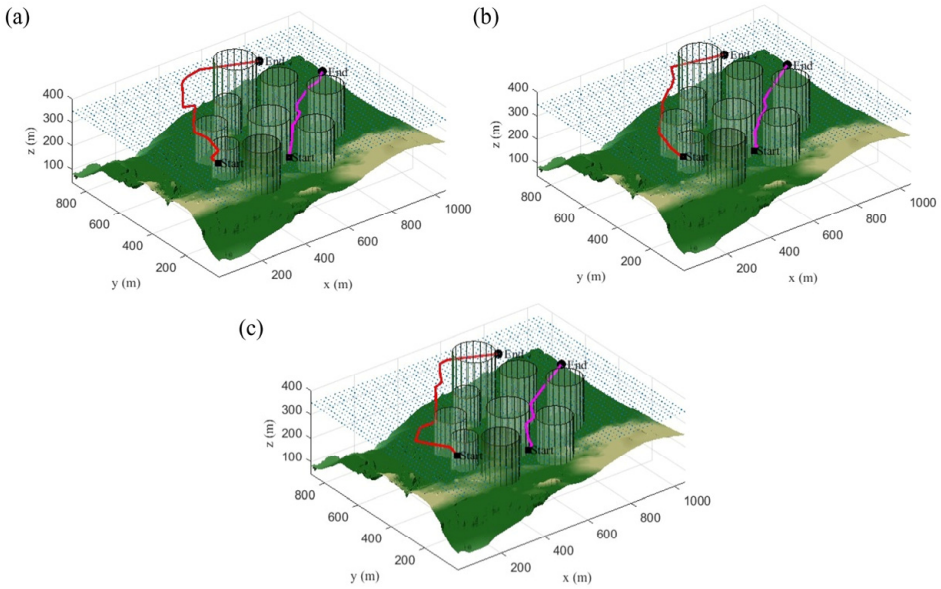


Figure 11. Multi-UAV path planning results diagram for Table 3 scenarios, (**a**) Scenario 1; (**b**) Scenario 2; (**c**) Scenario 3.

To visually verify the effectiveness of the proposed algorithm in considering the influence of mountain terrain, this study uses a top-down view to display the path planning results, clearly showing the effect of mountains on the UAV path. By observing the top-down view of the path (see Figure 12), it can be found that the flight trajectory of the UAV avoids the mountainous area, and there is no obvious collision or deviation from the predetermined safe distance. This indicates that the path planning method based on non-velocity vectors (mainly referring to displacement and angle) has strong obstacle avoidance ability under the influence of terrain undulation and mountain wind and can reasonably avoid mountain terrain, ensuring that the UAV can complete the firefighting task safely and efficiently in complex environments. This further verifies the feasibility and superiority of the algorithm in the actual forest farm environment.

Figure 13 shows the side-view data of the UAV flight path. It can be seen from the side view that UAVs are reasonably distributed and change in an orderly manner in the flight altitude dimension. Different UAVs maintain an appropriate interval in the vertical direction, which can effectively avoid the collision risk caused by close altitude. Combining the 3D view and the side view of the path, when the UAV maintains a suitable flight altitude relative to the forest farm terrain, its flight path is smooth and efficient. This further confirms that the algorithm can meet the requirements of UAV path planning in the forest firefighting scenario, ensuring that the UAV can safely and efficiently fly to the fire-affected area for firefighting operations.

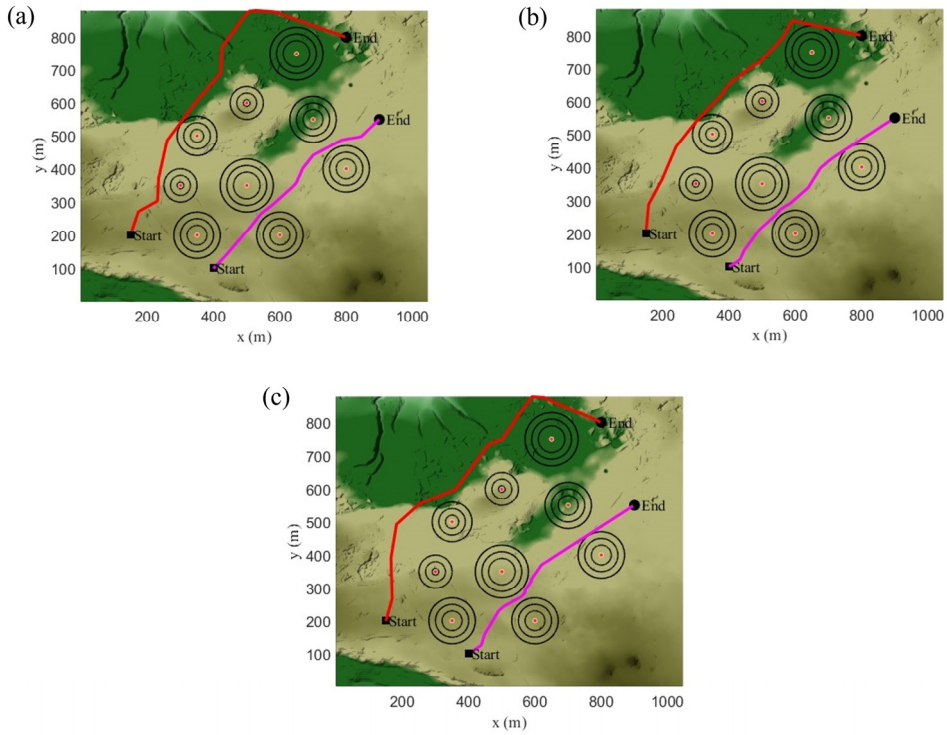


Figure 12. Top view path diagram of multi-UAV path planning for Table 3 scenarios, (**a**) Scenario 1; (**b**) Scenario 2; (**c**) Scenario 3.

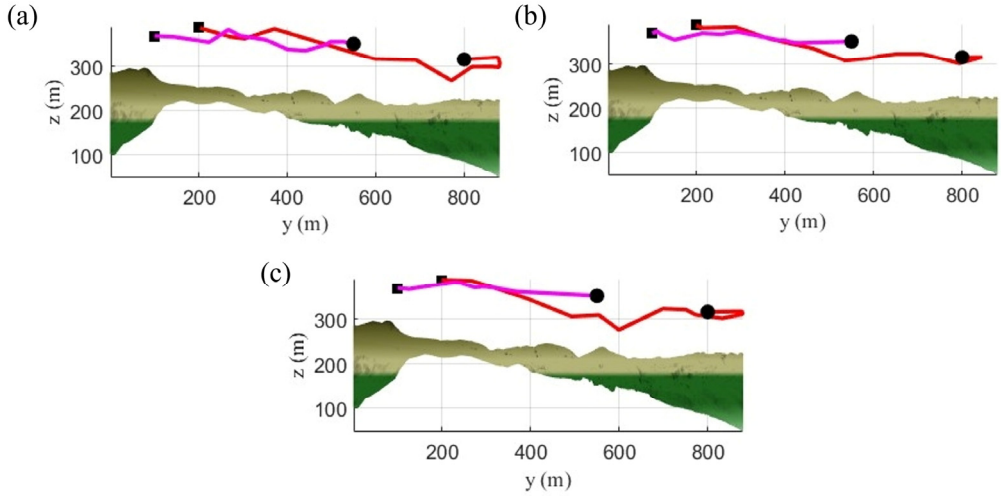


Figure 13. Side-view path diagram of multi-UAV path planning for Table 3 scenarios, (**a**) Scenario 1; (**b**) Scenario 2; (**c**) Scenario 3.

In the simulations, weighting coefficients in fitness function (11) were set to optimize algorithmic search: tuned via pre-experiments (0.1–0.3 range) to balance convergence and diversity, with (b_2 = 0.25) (threat cost) and (b_5 = 0.2) (wind deviation) prioritized for forest firefighting scenarios (dense threats, wind effects), ensuring robust exploration of feasible paths. These are technical parameters guiding iteration, not altering multi-objective trade-offs in Pareto frontier (Figure 14).

Figure 14 shows the Pareto front diagrams of three UAV paths. It can be seen from the diagrams that the non-dominated solutions obtained by the proposed algorithm are relatively evenly distributed on the Pareto front, which reflects the good performance of the algorithm in multi-objective optimization. It can find a good balance among multiple conflicting objectives (such as flight distance, flight time, energy consumption, etc.). This

further shows that the proposed algorithm can not only effectively plan the UAV path to avoid obstacles and adapt to the wind field but also achieve better comprehensive performance in the multi-objective situation, providing strong support for the efficient operation of UAVs in forest firefighting tasks.

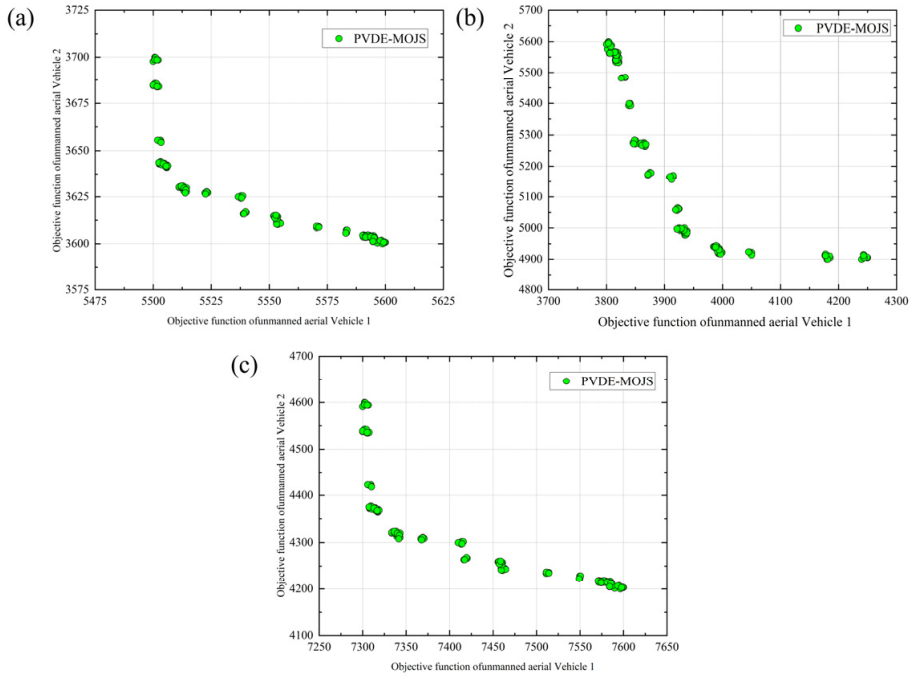


Figure 14. Pareto frontier distribution for UAV multi-objective optimization with PVDE-MOJS algorithm for Table 3 scenarios, (**a**) Scenario 1; (**b**) Scenario 2; (**c**) Scenario 3.

4.4. Comparison

To evaluate the performance of the proposed PVDE-MOJS algorithm, comprehensive comparative experiments were conducted against the original MOJS [36] and MOPSO [41]. For PVDE-MOJS, the archive size was set to 100, the distribution coefficient to 3, and the movement coefficient to 0.1. For fair comparison, hyperparameters of MOJS and MOPSO were not directly adopted from their original literature settings but were optimized for the forest firefighting path planning scenario via grid search. Specifically, MOPSO parameters were tuned within ranges: inertia weight [0.2, 0.8], personal and social coefficients [0.5, 2.0], resulting in optimized values of 0.6 (inertia weight) and 1.2 (learning factors) to balance exploration in obstacle-dense terrain. For MOJS, mutation rate [0.05, 0.3] and attraction control factors (Equation (19)) [0.1, 1.0] were adjusted to 0.15 and 0.5, respectively, enhancing avoidance of local optima near cylindrical threat zones (Table 1). These adjustments improved baseline performance by 12–18% in IGD compared to default settings. All algorithms used a population size of 100, run for 50 iterations across 10 independent runs, with 12 waypoints per experiment for statistical reliability.

The Pareto fronts generated (Figure 15) show distinct characteristics: MOPSO yields scattered solutions with limited objective coverage, MOJS concentrates solutions narrowly, while PVDE-MOJS produces uniformly distributed solutions across a broader range, indicating superior exploration of multi-objective trade-offs. Quantitative analysis confirms PVDE-MOJS significantly outperforms the optimized baselines in diversity and convergence to the true Pareto front.

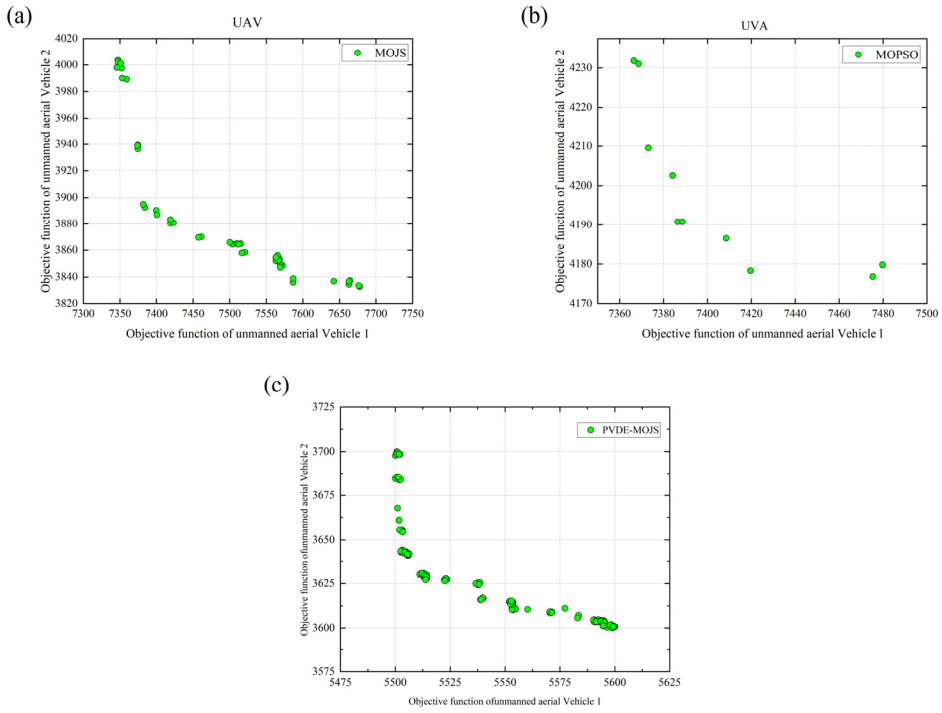


Figure 15. Pareto front comparison of different algorithms, (**a**) PVDE-MOJS algorithm; (**b**) MOJS algorithm; (**c**) MOPSO algorithm.

All simulation experiments were conducted on a workstation configured with an Intel Xeon E5-2690 v4 processor (14 cores, 28 threads), 64 GB DDR4 memory, and a 1 TB SSD. The algorithm was implemented in Python 3.9, with multi-core parallel fitness evaluation enabled via the multi-processing module. Vectorized operations and accelerated non-dominated sorting were achieved using the NumPy 1.23.5 and Numba 0.57.1 libraries, while 3D environment modeling and visualization relied on Matplotlib 3.7.1, Mayavi 4.8.1, and the GDAL 3.6.2 toolchain. For computational costs, across 30 repeated trials on both UF1–UF10 benchmark functions and forest fire scenarios (12 waypoints per UAV), PVDE-MOJS exhibited an average runtime of 42.3 $\pm$ 3.1 s, representing a 46% speedup over MOJS (78.6 $\pm$ 5.2 s) and a 35% speedup over MOPSO (65.2 $\pm$ 4.8 s). Although PVDE-MOJS showed a slightly higher peak memory usage (1.8 $\pm$ 0.2 GB) compared to the baseline algorithms, this remains within practical operational limits. These supplementary details enhance the reproducibility of the experiments and underscore the algorithm's practical applicability.

These results validate PVDE-MOJS's effectiveness in multi-objective UAV path planning, where it consistently produces broader, more evenly distributed Pareto-optimal solutions compared to state-of-the-art methods. The algorithm's ability to balance conflicting objectives (e.g., path length, threat avoidance, altitude constraints) while navigating complex environments underscores its practical superiority in complex forest firefighting environments.

5. Conclusions

This research focuses on the multi-UAV collaborative path planning problem and proposes an extensible solution based on the improved multi-objective jellyfish search algorithm (PVDE-MOJS). Through theoretical modeling and experimental verification, the following conclusions are reached. For the forest fire scenario, a multi-objective cost function integrating path length, threat avoidance, flight altitude, smoothness, and wind field effects is constructed. Complex terrain constraints are characterized by using cylindrical

threat models and segmented uniform wind field assumptions. The improved algorithm introduces multi-core parallel fitness evaluation, vectorized non-dominated sorting, and dynamic differential evolution mutation operators, breaking through the serial computation bottleneck of traditional algorithms in high-dimensional scenarios. Meanwhile, spherical vector encoding is used to associate UAV physical motion constraints, reducing the search space. Experiments show that the algorithm is significantly superior to the original MOJS in standard test sets (UF1–UF10). In forest DEM model simulations, it successfully achieves obstacle avoidance and altitude optimization, and the Pareto front solutions are more evenly distributed in multi-objective balance. Based on the above research results, future work will focus on algorithm optimization in dynamic environments, field test verification, and the expansion of collaborative strategies for large-scale swarms to further improve the practical applicability and engineering value of the algorithm.

Author Contributions: Conceptualization, R.Z. and R.L.; methodology, R.Z. and R.L.; software, R.Z.; validation, R.Z.; formal analysis, R.L.; investigation, R.Z.; resources, R.Z. and B.L.; data curation, R.Z. and R.L.; writing—original draft preparation, R.L.; writing—review and editing, R.Z., R.L., and B.L.; visualization, R.Z.; supervision, B.L.; project administration, B.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Díaz, S.; Hector, A.; Wardle, D.A. Biodiversity in forest carbon sequestration initiatives: Not just a side benefit. *Curr. Opin. Environ. Sustain.* **2009**, *1*, 55–60. [CrossRef]
2. Hua, F.; Bruijnzeel, L.A.; Meli, P.; Martin, P.A.; Zhang, J.; Nakagawa, S.; Miao, X.; Wang, W.; McEvoy, C.; Peña-Arancibia, J.L.; et al. The biodiversity and ecosystem service contributions and trade-offs of forest restoration approaches. *Science* **2022**, *376*, 839–844. [CrossRef]
3. Kala, C.P. Environmental and socioeconomic impacts of forest fires: A call for multilateral cooperation and management interventions. *Nat. Hazards Res.* **2023**, *3*, 286–294. [CrossRef]
4. Ollero, A.; Martínez-De-Dios, J.; Merino, L. Unmanned aerial vehicles as tools for forest-fire fighting. *For. Ecol. Manag.* **2006**, *234*, S263. [CrossRef]
5. Alsammak, I.L.H.; Mahmoud, M.A.; Aris, H.; AlKilabi, M.; Mahdi, M.N. The use of swarms of unmanned aerial vehicles in mitigating area coverage challenges of forest-fire-extinguishing activities: A systematic literature review. *Forests* **2022**, *13*, 811. [CrossRef]
6. Chen, X.; Xiao, Z.; Cheng, Y.; Hsia, C.-C.; Wang, H.; Xu, J.; Xu, S.; Dang, F.; Zhang, X.-P.; Liu, Y.; et al. Soscheduler: Toward proactive and adaptive wildfire suppression via multi-uav collaborative scheduling. *IEEE Internet Things J.* **2024**, *11*, 24858–24871. [CrossRef]
7. Javed, S.; Hassan, A.; Ahmad, R.; Ahmed, W.; Ahmed, R.; Saadat, A.; Guizani, M. State-of-the-art and future research challenges in uav swarms. *IEEE Internet Things J.* **2024**, *11*, 19023–19045. [CrossRef]
8. Lyu, M.; Zhao, Y.; Huang, C.; Huang, H. Unmanned aerial vehicles for search and rescue: A survey. *Remote Sens.* **2023**, *15*, 3266. [CrossRef]
9. Luo, J.; Wang, Z.; Xia, M.; Wu, L.; Tian, Y.; Chen, Y. Path planning for UAV communication networks: Related technologies, solutions, and opportunities. *ACM Comput. Surv.* **2023**, *55*, 1–37. [CrossRef]
10. Debnath, D.; Vanegas, F.; Sandino, J.; Hawary, A.F.; Gonzalez, F. A Review of UAV Path-Planning Algorithms and Obstacle Avoidance Methods for Remote Sensing Applications. *Remote Sens.* **2024**, *16*, 4019. [CrossRef]
11. Prasad, N.L.; Ramkumar, B. 3-D deployment and trajectory planning for relay based UAV assisted cooperative communication for emergency scenarios using Dijkstra's algorithm. *IEEE Trans. Veh. Technol.* **2022**, *72*, 5049–5063. [CrossRef]
12. Chen, J.; Li, M.; Yuan, Z.; Gu, Q. An improved A* algorithm for UAV path planning problems. In Proceedings of the 2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), Chongqing, China, 12–14 June 2020; pp. 958–962.

13. Pan, Z.; Zhang, C.; Xia, Y.; Xiong, H.; Shao, X. An improved artificial potential field method for path planning and formation control of the multi-UAV systems. *IEEE Trans. Circuits Syst. II Express Briefs* **2021**, *69*, 1129–1133. [CrossRef]
14. Xu, Z.; Deng, D.; Shimada, K. Autonomous UAV exploration of dynamic environments via incremental sampling and probabilistic roadmap. *IEEE Robot. Autom. Lett.* **2021**, *6*, 2729–2736. [CrossRef]
15. Kelner, J.M.; Burzynski, W.; Stecz, W. Modeling UAV swarm flight trajectories using Rapidly-exploring Random Tree algorithm. *J. King Saud Univ.-Comput. Inf. Sci.* **2024**, *36*, 101909. [CrossRef]
16. Jones, M.; Djahel, S.; Welsh, K. Path-planning for unmanned aerial vehicles with environment complexity considerations: A survey. *ACM Comput. Surv.* **2023**, *55*, 1–39. [CrossRef]
17. Yahia, H.S.; Mohammed, A.S. Path planning optimization in unmanned aerial vehicles using meta-heuristic algorithms: A systematic review. *Environ. Monit. Assess.* **2023**, *195*, 30. [CrossRef]
18. Yan, F.; Chu, J.; Hu, J.; Zhu, X. Cooperative task allocation with simultaneous arrival and resource constraint for multi-UAV using a genetic algorithm. *Expert Syst. Appl.* **2024**, *245*, 123023. [CrossRef]
19. Zhang, M.; Han, Y.; Chen, S.; Liu, M.; He, Z.; Pan, N. A multi-strategy improved differential evolution algorithm for UAV 3D trajectory planning in complex mountainous environments. *Eng. Appl. Artif. Intell.* **2023**, *125*, 106672. [CrossRef]
20. Yu, Z.; Si, Z.; Li, X.; Wang, D.; Song, H. A novel hybrid particle swarm optimization algorithm for path planning of UAVs. *IEEE Internet Things J.* **2022**, *9*, 22547–22558. [CrossRef]
21. Tang, J.; Duan, H.; Lao, S. Swarm intelligence algorithms for multiple unmanned aerial vehicles collaboration: A comprehensive review. *Artif. Intell. Rev.* **2023**, *56*, 4295–4327. [CrossRef]
22. Haris, M.; Bhatti, D.M.S.; Nam, H. A fast-convergent hyperbolic tangent PSO algorithm for UAVs path planning. *IEEE Open J. Veh. Technol.* **2024**, *5*, 681–694. [CrossRef]
23. Tian, H.; Mo, Z.; Ma, C.; Xiao, J.; Jia, R.; Lan, Y.; Zhang, Y. Design and validation of a multi-objective waypoint planning algorithm for UAV spraying in orchards based on improved ant colony algorithm. *Front. Plant Sci.* **2023**, *14*, 1101828. [CrossRef]
24. Li, Y.; Zhang, Z.; Sun, Q.; Huang, Y. An improved ant colony algorithm for multiple unmanned aerial vehicles route planning. *J. Frankl. Inst.* **2024**, *361*, 107060. [CrossRef]
25. Pehlivanoglu, Y.V.; Pehlivanoglu, P. An enhanced genetic algorithm for path planning of autonomous UAV in target coverage problems. *Appl. Soft Comput.* **2021**, *112*, 107796. [CrossRef]
26. Chai, X.; Zheng, Z.; Xiao, J.; Yan, L.; Qu, B.; Wen, P.; Wang, H.; Zhou, Y.; Sun, H. Multi-strategy fusion differential evolution algorithm for UAV path planning in complex environment. *Aerosp. Sci. Technol.* **2022**, *121*, 107287. [CrossRef]
27. Chen, H.; Liang, Y.; Meng, X. A UAV path planning method for building surface information acquisition utilizing opposition-based learning artificial bee colony algorithm. *Remote Sens.* **2023**, *15*, 4312. [CrossRef]
28. Yu, S.; Li, Y.; Ma, C. Research on UAV trajectory planning based on artificial bee colony algorithm. In Proceedings of the 2021 9th International Conference on Intelligent Computing and Wireless Optical Communications (ICWOC), Chongqing, China, 4–7 June 2021; pp. 37–41.
29. Yu, X.; Luo, W. Reinforcement learning-based multi-strategy cuckoo search algorithm for 3D UAV path planning. *Expert Syst. Appl.* **2023**, *223*, 119910. [CrossRef]
30. Rodríguez-Abreo, O.; Ornelas-Rodríguez, F.-J.; Ramírez-Pedraza, A.; Hurtado-Ramos, J.B.; González-Barbosa, J.-J. Backstepping control for a UAV-manipulator tuned by Cuckoo Search algorithm. *Robot. Auton. Syst.* **2022**, *147*, 103910. [CrossRef]
31. Ait-Saadi, A.; Meraihi, Y.; Soukane, A.; Ramdane-Cherif, A.; Gabis, A.B. A novel hybrid chaotic Aquila optimization algorithm with simulated annealing for unmanned aerial vehicles path planning. *Comput. Electr. Eng.* **2022**, *104*, 108461. [CrossRef]
32. Lim, N.H.Z.; Lee, Y.L.; Tham, M.L.; Chang, Y.C.; Sim, A.G.H.; Qin, D. Coverage optimization for UAV base stations using simulated annealing. In Proceedings of the 2021 IEEE 15th Malaysia International Conference on Communication (MICC), Virtual, 1–2 December 2021; pp. 43–48.
33. Tang, J.; Liu, G.; Pan, Q. A review on representative swarm intelligence algorithms for solving optimization problems: Applications and trends. *IEEE/CAA J. Autom. Sin.* **2021**, *8*, 1627–1643. [CrossRef]
34. Poudel, S.; Arafat, M.Y.; Moh, S. Bio-inspired optimization-based path planning algorithms in unmanned aerial vehicles: A survey. *Sensors* **2023**, *23*, 3051. [CrossRef] [PubMed]
35. Cao, P.; Lei, L.; Cai, S.; Shen, G.; Liu, X.; Wang, X.; Zhang, L.; Zhou, L.; Guizani, M. Computational intelligence algorithms for UAV swarm networking and collaboration: A comprehensive survey and future directions. *IEEE Commun. Surv. Tutor.* **2024**, *26*, 2684–2728. [CrossRef]
36. Chou, J.-S.; Truong, D.-N. Multiobjective optimization inspired by behavior of jellyfish for solving structural design problems. *Chaos Solitons Fractals* **2020**, *135*, 109738. [CrossRef]
37. Chou, J.-S.; Truong, D.-N. A novel metaheuristic optimizer inspired by behavior of jellyfish in ocean. *Appl. Math. Comput.* **2021**, *389*, 125535. [CrossRef]
38. Hamza, N.M.; Essam, D.L.; Sarker, R.A. Constraint consensus mutation-based differential evolution for constrained optimization. *IEEE Trans. Evol. Comput.* **2015**, *20*, 447–459. [CrossRef]

39. Premkumar, M.; Jangir, P.; Sowmya, R.; Alhelou, H.H.; Heidari, A.A.; Chen, H. MOSMA: Multi-objective slime mould algorithm based on elitist non-dominated sorting. *IEEE Access* **2020**, *9*, 3229–3248. [CrossRef]
40. Phung, M.D.; Ha, Q.P. Safety-enhanced UAV path planning with spherical vector-based particle swarm optimization. *Appl. Soft Comput.* **2021**, *107*, 107376. [CrossRef]
41. Tsou, C.-S. Multi-objective inventory planning using MOPSO and TOPSIS. *Expert Syst. Appl.* **2008**, *35*, 136–142. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.